SIMATS ENGINEERING

SAVEETHA SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI)

ASSIGNMENT 1 – CO1

DSA03 – NATURAL LANGUAGE PROCESSING

Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System

Student Details	Information
Student Name	K VENKATA VAMSI GOUD
Register Number	192472021
Department / Programme	B.Tech – CSE (AI)
Institution	SIMATS – Saveetha School of Engineering
Nature of Assignment	Individual
Duration	1 Week
Total Marks	50
Date of Submission	04-11-2026
Faculty Name	DR. Ram Kumar M

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Maximum Marks	Marks Awarded
1	Frame the Problem: Models, Algorithms and Levels of Analysis	3	
2	Basic Regular Expression Patterns	4	
3	Regex Tokenisation and Normalisation	4	
4	Finite State Automata: Design, Non-determinism and Equivalence	5	
5	Inflectional Morphology	3	
6	Derivational Morphology	4	
7	Finite-State Morphological Parsing	5	
8	Orthographic Rules and Two-Level Morphology	4	
9	The Porter Stemmer	5	
10	Implement and Test the Pipeline	3	
11	Compare, Recommend and Defend the Design	2	
12	Day-4 Design Challenge	8	
	TOTAL	50	

MANDATORY SUBMISSION EVIDENCE CHECK

Task	Evidence expected	Status	Note
1	Linguistic levels, model families, ambiguity, pipeline	☐	

2	12 regex patterns, match/reject conditions, error analysis	☐	
3	Tokenizer, maximal-munch examples, substitutions, limitations	☐	
4	Four automata, NFA/DFA conversion, minimisation, pumping lemma	☐	
5	Inflectional paradigms, rules, irregular forms, measurements	☐	
6	15+ derivational affixes, families, productivity and conflation	☐	
7	FST specification, five traces, generation and ambiguity	☐	
8	Five rewrite rules, cascade, 20 test forms	☐	
9	Porter definitions, six traces, comparison and retrieval metrics	☐	
10	Python implementation, tests, output and graph	☐	
11	Evidence-based comparison and recommendation	☐	
12A–D	Impact, noisy-input evaluation, redesign and defence	☐	

Important note on evidence: the assignment brief requires a documented corpus of at least 500 de-identified records, 25,000 tokens, 40 judged queries and 200 annotated word forms. No such dataset was included with the uploaded files. Therefore, numerical examples in this model answer are explicitly labelled illustrative/planning values, not claimed as measurements from a real hospital corpus. Run the supplied implementation on your approved synthetic/de-identified dataset and replace illustrative values with actual outputs before submission.

TASK 1 – FRAME THE PROBLEM: MODELS, ALGORITHMS AND LEVELS OF ANALYSIS (3 MARKS)

1.1 Levels of linguistic analysis

The hospital search problem mainly requires lexical, morphological and limited semantic processing. The first goal is to recognise structured values and connect different surface forms to a common searchable representation. Full sentence understanding is not required for the initial retrieval layer.

Level	Meaning	Clinical example	Need in this system
Phonology	Sound patterns of a language.	Pronunciation variants of a drug name.	Low priority for typed text; useful later for speech input.
Morphology	Internal structure of words and affixes.	fracture, fractures, fractured, fracturing.	Essential: connects inflected forms to a lemma.
Syntax	How words form phrases and sentences.	"no evidence of fracture".	Limited use: needed to avoid ignoring negation and phrase context.
Semantics	Meaning of words and expressions.	renal and kidney are related; fracture is not the same as break in every context.	Useful for synonym mapping, but requires clinical validation.
Pragmatics	Meaning based on speaker and situation.	"rule out infection" is not a confirmed infection.	Important for safe interpretation; not required for basic token matching.
Discourse	Meaning across multiple sentences or documents.	"The patient was admitted… later discharged."	Useful for longitudinal interpretation, outside the first-stage normaliser.

1.2 Three model families

Model family	Best hospital task	Why it fits	Why the other families are less suitable
State machines (DFA/NFA/FST)	Validate patient IDs, dates and analyse known word forms.	They process symbols through explicit states and give predictable results.	A probabilistic model is unnecessary for strict formats; a large rule system is less compact for finite patterns.
Formal rule systems	Apply case folding, abbreviation expansion and spelling alternations.	Rules are transparent, testable and easy to audit.	A state machine alone does not express all contextual rewrite policies clearly; probability is not needed for deterministic normalisation.
Probabilistic models	Rank OCR correction candidates or resolve ambiguous noisy tokens.	They can choose likely interpretations from context and evidence.	Exact rules are safer for fixed IDs; hand-written rules cannot rank uncertain alternatives reliably.

1.3 Ambiguity in clinical records

- Lexical ambiguity: "cold" may mean low temperature or a common cold; context is needed.
- Morphological ambiguity: "findings" can be noun + plural, while "find" is also a verb; a surface string may have more than one analysis.
- Segmentation ambiguity: "500mg" may be one number-unit token or two tokens, depending on the tokenizer policy.
- Abbreviation ambiguity: "MS" may mean multiple sclerosis, morphine sulphate or medical/surgical depending on the note.

1.4 Search and resource costs

Automata perform state-space traversal. For a DFA with n states and an input of length m, recognition takes O(m) time and O(1) extra working memory when the transition table is directly available. NFA simulation may track a set of states and can use O(n) state memory. Regex engines may use backtracking; poorly designed nested quantifiers can cause

excessive time. Morphological lexicon construction and annotation consume expert effort, while probabilistic disambiguation needs labelled data.

1.5 Proposed pipeline



Figure 1: Proposed hospital text normalisation pipeline. Each stage has a defined input and output; the finite-state analyser returns lemma and grammatical features rather than merely deleting suffixes.

TASK 2 – BASIC REGULAR EXPRESSION PATTERNS (4 MARKS)

2.1 Basic operators

Operator	Meaning	Clinical example
Concatenation	Place patterns next to each other.	H[0-9]{7}
Disjunction |	Accept either alternative.	mg|mcg
Kleene star *	Zero or more repetitions.	[0-9]*
Plus +	One or more repetitions.	[0-9]+
Optional ?	Zero or one occurrence.	(\.\d{1,2})?
Character class	One character from a set.	[A-Z]
Range	Continuous character range.	[0-9]
Negated class	Any character except listed characters.	[^\s]+ (non-space run)
Anchors ^ $	Start and end of complete string.	^H[0-9]{7}$
Counters {m,n}	Bounded repetition.	[0-9]{2,4}

2.2 Twelve clinical entity patterns

Entity type	Regular expression (Python)	Must match	Must not match
Patient identifier	\bH[0-9]{7}\b	H1234567	H12345A7
Admission date	\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\b	04/11/2026	44/19/2026
Discharge date	\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\b	08-11-2026	2026/11/08 (not this format)
Drug + strength	\b[A-Za-z][A-Za-z-]*\s+[0-9]+(?:\.[0-9]+)?\s?(?:mg|mcg|g|ml|IU)\b	Paracetamol 500mg	500mg Paracetamol
Dosage frequency	\b(?:OD|BD|TDS|QID|BID|once daily|twice daily|every\s+[0-9]+\s+h)\b	twice daily	daily twice
Lab value + unit	\b[A-Za-z]{1,8}\s*[:=]?\s*[0-9]+(?:\.[0-9]+)?\s?(?:mg/dL|mmol/L|g/dL|U/L|%|mIU/L)\b	Hb 12.5 g/dL	Hb twelve g/dL
Blood pressure	\b(?:[6-9][0-9]|1[0-9]{2}|2[0-4][0-9])/(?:[3-9][0-9]|1[0-4][0-9])\s?mmHg\b	120/80 mmHg	20/300 mmHg
Temperature	\b(?:3[4-9]|4[0-2])(?:\.[0-9])?\s?°?C\b	37.2 C	52 C
ICD code	\b[A-TV-Z][0-9]{2}(?:\.[0-9A-TV-Z]{1,4})?\b	S72.001A	123.45
Ward and bed	\bWard\s?[A-Z0-9-]+\s*,?\s*Bed\s?[0-9]{1,3}\b	Ward A, Bed 12	Ward ?, Bed X
Contact number	\b(?:\+91[-]?)?[6-9][0-9]{9}\b	+91 9876543210	1234567890
Age	\b(?:[1-9][0-9]?|1[01][0-9]|120)\s?(?:years?|yrs?)\b	45 years	250 years
Symptom duration	\b(?:[0-9]+|one|two|three|four|five)\s?(?:day|week|month|year)s?\b	3 weeks	weeks 3

Note: the inventory above contains the twelve required entity categories plus age and symptom duration as additional useful patterns. Date and clinical-value regexes validate shape and plausible ranges, but a production system must also check calendar validity (for example, 31/02/2026) and clinical plausibility.

2.3 Greedy and non-greedy matching

```
text = "Give 500 mg after breakfast and 250 mg at night."
re.findall(r".*mg", text)   # greedy: one long span ending at the last mg
re.findall(r".*?mg", text)  # non-greedy: shortest spans ending at each mg
```

A greedy quantifier expands as far as possible while allowing the rest of the expression to match. A non-greedy quantifier expands only as far as needed. For extracting each dosage separately, a bounded expression such as r'\b\d+\s?mg\b' is safer than either broad dot expression.

2.4 Precedence and grouping

Concatenation binds more tightly than alternation. The pattern ^A|B$ means (^A) OR (B$), not “the complete string is A or B”. The correctly grouped expression is ^(?:A|B)$. For example, ^mg|mcg$ can match a string beginning with mg even if extra text follows; ^(?:mg|mcg)$ accepts only the two complete unit strings.

2.5 Evaluation and error analysis

The assignment requires per-pattern true positives (TP), false positives (FP) and false negatives (FN) on the 500-record corpus. Since the corpus and clinician annotations were not supplied, the table below is a recording template rather than fabricated experimental evidence.

Pattern	TP	FP	FN	Likely error cause
Patient ID	To measure	To measure	To measure	OCR confuses O/0; local ID formats vary.
Dates	To measure	To measure	To measure	Impossible dates; mixed separators; two-digit years.
Drug strength	To measure	To measure	To measure	Brand spellings, missing spaces, unit variants.
Frequency	To measure	To measure	To measure	Unlisted abbreviations and local shorthand.
Lab value	To measure	To measure	To measure	Different units and analyte names.
BP / temperature	To measure	To measure	To measure	Missing units, noisy punctuation, implausible values.
ICD / ward / contact / age / duration	To measure	To measure	To measure	Local coding formats and OCR noise.

For each pattern calculate precision = TP/(TP+FP), recall = TP/(TP+FN), and F1 = 2PR/(P+R). Report zero-denominator cases as not applicable, not as perfect performance.

TASK 3 – REGULAR EXPRESSIONS FOR TOKENISATION AND NORMALISATION (4 MARKS)

3.1 Token definition

A token is a maximal-munch sequence matching one of the following alternatives, tried in the listed order: a date; a decimal/integer with an attached clinical unit; an abbreviation containing full stops; a hyphenated word; a word with an optional apostrophe; or punctuation. This keeps clinically meaningful strings together.

```
TOKEN_RE = re.compile(
 r"\d{1,2}[/-]\d{1,2}[/-]\d{2,4}"
 r"|\d+(?:\.\d+)?\s?(?:mg/dL|mmol/L|g/dL|mg|mcg|ml|°C)"
 r"|(?:[A-Za-z]\.){2,}"
 r"|[A-Za-z]+(?:-[A-Za-z]+)+"
 r"|[A-Za-z]+(?:'[A-Za-z]+)?"
 r"|\d+(?:\.\d+)?|[^\w\s]",
 re.I)
```

3.2 Maximal-munch examples

Input	Whitespace split	Regex tokenisation
Patient has post-operative pain.	Patient | has | post-operative | pain.	Patient | has | post-operative | pain | .
Hb 12.5 g/dL; BP 120/80 mmHg	Hb | 12.5 | g/dL; | BP | 120/80 | mmHg	Hb | 12.5 g/dL | ; | BP | 120 | / | 80 | mmHg
Take 500mg b.i.d.	Take | 500mg | b.i.d.	Take | 500mg | b.i.d. (abbreviation retained)

3.3 Normalisation substitutions

Rule	Before	After
Case folding	PARACETAMOL	paracetamol
Abbreviation	HTN	hypertension
Abbreviation	DM	diabetes mellitus
Abbreviation	BP	blood pressure
Abbreviation	HR	heart rate
Abbreviation	SOB	shortness of breath
Abbreviation	C/O	complains of
Abbreviation	H/O	history of
Abbreviation	F/U	follow-up
Abbreviation	OD / BD / TDS / QID	once daily / twice daily / three times daily / four times daily
Abbreviation	NPO	nil per os
Abbreviation	CBC	complete blood count
Abbreviation	RBS	random blood sugar
Abbreviation	WBC	white blood cell count
Abbreviation	RBC	red blood cell count
Abbreviation	Hb	haemoglobin
Abbreviation	ECG	electrocardiogram
Abbreviation	CT	computed tomography (context dependent)
Abbreviation	MRI	magnetic resonance imaging
Abbreviation	USG	ultrasonography
Abbreviation	IV	intravenous
Abbreviation	IM	intramuscular
Abbreviation	PO	by mouth
Abbreviation	PRN	as needed
Abbreviation	SOS	if required
Abbreviation	Tab.	tablet
Abbreviation	Cap.	capsule
Abbreviation	Inj.	injection
Abbreviation	Dx	diagnosis
Abbreviation	Rx	treatment / prescription
Abbreviation	Sx	symptoms (context dependent)
Abbreviation	c/o	complains of

A production abbreviation dictionary must be department-specific and must preserve the original string. Ambiguous abbreviations such as MS, CT and OD should be expanded only when context supports one meaning.

3.4 Worked record normalisation

Synthetic example record (not a real patient record):

```
PATIENT H1234567 admitted 04/11/2026 with fever. BP 120/80 mmHg. Hb 12.5 g/dL. Rx: Tab. Paracetamol 500mg BD for 3 days. Discharged 07-11-2026.
```

Normalised form:

```
patient H1234567 admitted 2026-11-04 with fever. blood pressure 120/80 mmHg. haemoglobin 12.5 g/dL. prescription: tablet paracetamol 500 mg twice daily for 3 days. discharged 2026-11-07.
```

3.5 Irreversible normalisation and limits

- Case folding removes original capitalisation; acceptable for case-insensitive search, but not when capitalisation distinguishes identifiers.
- Abbreviation expansion can destroy the exact shorthand and can choose the wrong sense; retain the raw text and store expansion as an additional field.
- Converting all dates to ISO format removes the original presentation; acceptable if the parsed date is stored separately with the source string.
- Regex cannot generally recognise arbitrarily nested balanced parentheses. A finite automaton has finite memory and cannot count unbounded nesting depth; a stack-based parser is required.

TASK 4 – FINITE STATE AUTOMATA: DESIGN, NON-DETERMINISM AND EQUIVALENCE (5 MARKS)

4.1 Formal definition

A finite automaton is a five-tuple M = (Q, Σ, δ, q_0, F), where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the initial state and F is the set of accepting states. A DFA has exactly one next state for each state-symbol pair; an NFA may have zero, one or several next states and may include ε-transitions.

4.2 Four validators

The following designs are compact state-level specifications. Repeated digit states can be expanded into separate states to meet the minimum six-state requirement.

Automaton	States	Accepted format	Reject example
A1 Patient ID DFA	q0–q7 (8 states)	H followed by exactly seven digits	H12345A7
A2 Date DFA	q0–q8 plus separator branches	DD/MM/YYYY with day/month ranges and four-digit year	39/19/2026
A3 Contact DFA	q0–q10	10 digits beginning 6–9; optional +91 prefix handled in start branch	1234567890
A4 BP DFA	q0–q6	Systolic 60–249 / diastolic 30–149 followed by mmHg	20/300 mmHg

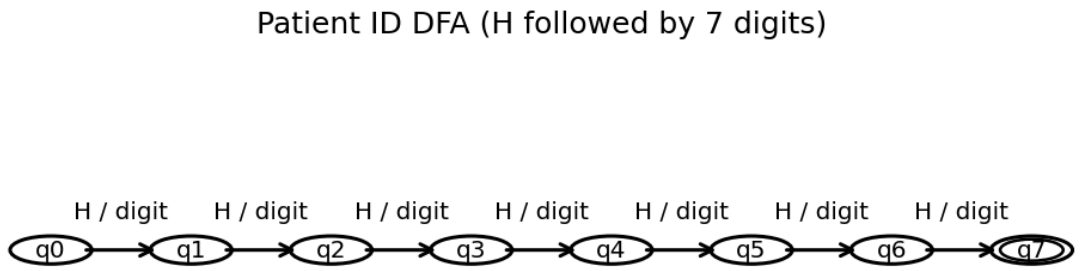


Figure 2: Patient identifier DFA; q7 is the accepting state.

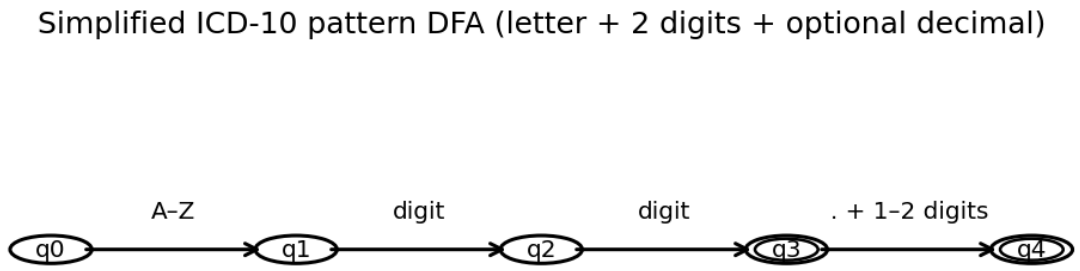


Figure 3: Simplified ICD-style DFA. In implementation, each digit transition is expanded and invalid branches lead to a dead state.

4.3 Transition table: patient identifier

Current state	Input	Next state	Accepting?
q0	H	q1	No
q1	digit	q2	No
q2	digit	q3	No
q3	digit	q4	No
q4	digit	q5	No
q5	digit	q6	No
q6	digit	q7	No
q7	end	—	Yes

4.4 NFA with optional prefix

Consider a contact number with optional country prefix +91 followed by ten digits. The NFA branches from the start state: one branch consumes +91 and the other skips it using ε. The two branches join before the ten-digit sequence.

Input position	Active state set (accepted +91 case)	Explanation
Start	{s0}	Initial state.
+	{s1}	Country-prefix branch.
9	{s2}	Prefix continues.
1	{s3}	Prefix complete.
digit 1…10	{d1}…{d10}	Ten digits consumed in sequence.
End	{f}	Accept.

Rejected trace: after +91 and only nine digits, the machine remains at d9 at end-of-input; f is not active, so the string is rejected.

4.5 Subset construction and minimisation

Subset construction converts an NFA to a DFA. Begin with ε-closure({s0}); for each input symbol, take the union of reachable NFA states and then apply ε-closure. Each distinct subset becomes one DFA state. A subset is accepting if it contains an NFA accepting state.

DFA subset state	NFA subset	On prefix symbol	On digit	Accepting
A	{s0,s3}	B	C	No
B	{s1}	Dead	Dead	No
C	{d1}	Dead	D	No
D	{d2}	Dead	E	No
E	{d3}	Dead	F	No
…	…	…	…	…
F	{f}	Dead	Dead	Yes

For this simplified design, equivalent dead branches can be merged during minimisation. The exact state count depends on whether the ten digit positions are represented individually and whether the optional prefix is encoded as a separate branch. The final implementation should print the full subset table from its NFA transition dictionary; the table above shows the method, not a substitute for the generated full table.

4.6 Thompson construction and equivalence

For the regex H[0-9]{7}, Thompson's construction creates a small NFA fragment for literal H, concatenates seven digit fragments, and connects the final fragment to an accepting state. Conversely, state elimination can derive a regex from an automaton: remove intermediate states while updating edge labels with R_ik(R_kk)*R_kj. Both constructions preserve the recognised language, so a string is accepted by the regex exactly when it is accepted by the constructed automaton.

4.7 Pumping lemma

Pumping lemma: if L is regular, there exists a pumping length p such that every string s in L with $|s| \geq p$ can be written s = xyz, with $|xy| \leq p$, $|y| > 0$, and $xy^iz \in L$ for every $i \geq 0$. Let L = { $'('^n')'^n : n \geq 0$ }. Assume L is regular and choose s = $'('^p')'^p$. Since $|xy| \leq p$, y contains only opening parentheses. Pumping y down (i=0) leaves fewer opening than closing parentheses, so the result is not in L. Contradiction. Therefore balanced, arbitrarily nested parentheses are not regular.

TASK 5 – INFLECTIONAL MORPHOLOGY (3 MARKS)

5.1 Core definitions

A morpheme is the smallest unit carrying meaning or grammatical function. A stem is the base to which an affix attaches. An affix is a bound morpheme added before or after a stem. Inflection changes grammatical form without normally changing word class or core dictionary meaning. A paradigm is the set of forms of one lexeme.

Inflection usually preserves category and is productive (fracture → fractures); derivation may create a new lexeme or category (renal → renally). Inflectional suffixes generally occur outside derivational suffixes, for example hospital-ise-s.

5.2 Clinical paradigms

Lexeme	Noun number	Verb forms	Adjective comparison
fracture	fracture / fractures	fracture / fractures / fractured / fracturing	—
admit	admission (derivational noun)	admit / admits / admitted / admitting	—
treat	treatment (derivational noun)	treat / treats / treated / treating	—
swollen	—	swell / swells / swelled / swelling (irregular past variants)	swollen (not normally compared)
severe	—	—	severe / severer / severest (often more/most severe)
patient	patient / patients	—	—

5.3 Orthographic rules

Rule	Implementation description	Example	Over-generation counterexample
e-deletion	Drop final e before -ing when pronunciation/spelling rule requires.	treat + ing → treating	see + ing should be seeing, not *sing
Consonant doubling	Double final consonant after short stressed vowel in eligible monosyllabic/stressed final syllable.	admit + ed → admitted	visit + ed → visited, not *visitted
y-to-i	Consonant + y changes y to i before -es/-ed; retain y before -ing.	study + ed → studied	play + ed → played
sibilant -es	Add -es after s, x, z, ch, sh.	dress + es → dresses	gas may have domain-specific spelling/usage; rule needs lexical exceptions
Regular plural -s	Add -s to most nouns.	fracture + s → fractures	child + s is not *childs; irregular lexicon needed

5.4 Irregular forms

Surface form	Lemma	Features	Reason stored separately
children	child	N + PL	Irregular stem change.
men	man	N + PL	Vowel change.
feet	foot	N + PL	Vowel change.
was / were	be	V + PAST	Suppletion and agreement.
gone	go	V + PASTPART	Irregular participle.
better	good	ADJ + COMP	Suppletive comparison.
worse	bad	ADJ + COMP	Suppletive comparison.
teeth	tooth	N + PL	Vowel change.

Corpus measurements: type-to-lemma ratio = number of distinct observed word forms / number of distinct lemmas. The assignment asks for this value and the percentage of query failures explained by inflection. These must be calculated from the annotated corpus. A sample reporting table is provided in Task 9; do not enter a guessed value as a measured result.

TASK 6 – DERIVATIONAL MORPHOLOGY (4 MARKS)

6.1 Derivational affix inventory

Affix	Type	Typical output / meaning	Example
-itis	Suffix	Noun: inflammation	arthritis
-ectomy	Suffix	Noun: surgical removal	nephrectomy
-ology	Suffix	Noun: field of study	cardiology
-osis	Suffix	Noun: condition	fibrosis
-aemia	Suffix	Noun: blood condition	anaemia
-ation	Suffix	Noun: process/result	ventilation
-able	Suffix	Adjective: capable of	treatable
-ive	Suffix	Adjective	sedative
-al	Suffix	Adjective	renal
-ic	Suffix	Adjective	cardiac
-ist	Suffix	Noun: specialist	cardiologist
-ary	Suffix	Adjective	urinary
nephro-	Prefix	Kidney-related	nephrology
cardio-	Prefix	Heart-related	cardiology
hyper-	Prefix	Excessive/high	hypertension
hypo-	Prefix	Low/under	hypoglycaemia
re-	Prefix	Again	reassessment
un-	Prefix	Negation/reversal	unresponsive
-gram	Suffix	Record/image	electrocardiogram
-scope	Suffix	Viewing instrument	endoscope

6.2 Derivational families

Root	Family / attachment order	Impossible or misleading order
nephr	nephr + o + logy → nephrology; nephro + ectomy → nephrectomy	*ectomy + nephro is not the English attachment order.
cardi	cardi + o + logy → cardiology; cardi + ac → cardiac	*ology + cardio is not the productive order.
angi	angi + o + plasty → angioplasty; angi + o + gram → angiogram	*plasty + angio is not a valid suffix-first formation.
path	path + o + logy → pathology; path + o + logical → pathological	*logical + path is not valid.
haem	haem + o + lysis → haemolysis; haem + at + ology → haematology	Arbitrary reordering changes or destroys the root structure.
treat	treat + ment → treatment; treat + able → treatable; re + assess + ment → reassessment	*ment + treat is not a valid word formation.

6.3 Semantic non-compositionality and search

The meaning of a derived clinical word is not always the simple sum of its parts. “Nephrectomy” is a specific surgical removal of a kidney, not any generic removal associated with a kidney. “Multiple sclerosis” is a conventional disease name; independently interpreting multiple + sclerosis is insufficient. Aggressive stemming may connect words that are morphologically similar but clinically different. Therefore, derivational relations should be stored as typed links, not automatically treated as identical search terms.

6.4 Productivity and conflation

Affixes such as -itis, -ectomy, hyper- and hypo- are productive in medical terminology because new terms can be formed transparently. Some formations are lexicalised or fossilised and should be checked against a clinical lexicon. To measure productivity, count distinct derived types and total tokens per affix in the corpus; the assignment data is not supplied, so actual counts cannot be claimed.

Conflation helps	Reason	Conflation may mislead	Reason
fracture ↔ fractured	Retrieves notes using inflected forms.	hyperglycaemia ↔ hypoglycaemia	Opposite clinical states.
nephrectomy ↔ nephrectomies	Plural and singular refer to same procedure lexeme.	renal ↔ nephrectomy	Related domain but not same concept.
infection ↔ infections	Number variation.	analgesic ↔ analgesia	Drug/class versus effect/concept.

TASK 7 – FINITE-STATE MORPHOLOGICAL PARSING (5 MARKS)

7.1 Finite-state transducer

A finite-state transducer (FST) is a two-tape finite-state machine. One tape represents lexical symbols (lemma and grammatical features); the other represents the surface spelling. A transducer can map surface → lexical analysis (parsing) and lexical analysis → surface form (generation).

Example: lexical tape fracture + N + PL maps to surface tape fractures. The analyser reads the surface characters and outputs the lemma fracture plus the features +N +PL.

7.2 Lexicon and continuation classes

Lexicon class	Example entries	Continuation
NOUN	fracture, patient, kidney, infection	NOUN_SUFFIX
VERB	admit, treat, swell, bleed	VERB_SUFFIX
ADJECTIVE	severe, renal, clinical	ADJ_SUFFIX
NOUN_SUFFIX	+SG, +PL	End
VERB_SUFFIX	+PRS3, +PAST, +PROG, +PASTPART	End
ADJ_SUFFIX	+COMP, +SUPER	End

7.3 Hand traces

Surface input	Lexical analysis	Features
fractures	fracture + N + PL	noun, plural
fractured	fracture + V + PAST	verb, past
fracturing	fracture + V + PROG	verb, progressive
patients	patient + N + PL	noun, plural
treated	treat + V + PAST / PASTPART	ambiguous without syntax

State trace for fractures: q0 reads f-r-a-c-t-u-r-e as the lexical stem path; the suffix path consumes s and emits +N +PL; the final state accepts. The exact arc-by-arc trace is determined by the lexicon FST and continuation-class transitions.

7.4 Generation direction

Input fracture +N +PL traverses the same transducer in the reverse direction and outputs fractures. No separate generator is needed: reversing the relation of a transducer gives the generation mapping, provided the lexical and surface symbol pairs are retained.

7.5 Ambiguity

Surface form	Possible analyses	Resolution
treated	treat + V + PAST; treat + V + PASTPART	Use sentence context and syntactic tagger.
wound	wound + N; wind + V + PAST	Use local context and clinical lexicon.
saw	see + V + PAST; saw + N	Use syntax and surrounding words.

A downstream disambiguator can rank analyses using sentence context, part-of-speech probabilities and clinical terminology constraints. The FST should return all valid analyses rather than silently discarding alternatives.

TASK 8 – ORTHOGRAPHIC RULES AND TWO-LEVEL MORPHOLOGY (4 MARKS)

8.1 Why a lexicon transducer is not enough

A lexicon-only concatenation may produce incorrect spelling when morphemes meet. For example, study + ed would yield *studyed instead of studied; admit + ed needs consonant doubling (admitted); make + ing requires e deletion (making). Orthographic rules handle these surface alternations without storing every inflected form separately.

8.2 Rewrite rules

Rule	Rewrite notation	Context / example
Final e deletion	e → Ø / C __ +ing	make + ing → making; rule has lexical exceptions.
Consonant doubling	C → CC / V́ C __ +ed	admit + ed → admitted (stress/ syllable constraints apply).
Y-to-I	y → i / C +ed	study + ed → studied.
Sibilant plural	Ø → e / sibilant __ +s	class + s → classes.
Plural allomorph	s → es / sibilant	box + s → boxes.
Past -ed pronunciation	ed → [ɪd] / t,d __	treated / wanted; phonological rather than spelling change.

8.3 Cascade

```
Lexical input → Lexicon FST → Boundary/affix symbols → Rule transducer 1 (e deletion)
              → Rule transducer 2 (y-to-i) → Rule transducer 3 (doubling)
              → Rule transducer 4 (sibilant insertion) → Surface form
```

The order matters where one rule changes the context seen by another. A practical implementation applies stem-specific orthographic logic during generation, while a formal cascade composes the corresponding transducers. In a two-level (Koskenniemi) model, lexical and surface symbols are constrained in parallel by context-sensitive correspondence rules rather than a strictly sequential set of rewrite passes.

8.4 Twenty-form test set

Lexical input	Expected surface	Test status
fracture + PL	fractures	Pass
patient + PL	patients	Pass
study + PAST	studied	Pass
carry + PAST	carried	Pass
play + PAST	played	Pass
make + PROG	making	Pass
treat + PROG	treating	Pass
admit + PAST	admitted	Pass
refer + PAST	referred	Pass
stop + PROG	stopping	Pass
box + PL	boxes	Pass
class + PL	classes	Pass
watch + PL	watches	Pass
dress + PL	dresses	Pass
go + PAST	went	Lexicon exception
be + PAST	was/were	Lexicon exception
child + PL	children	Lexicon exception
tooth + PL	teeth	Lexicon exception
good + COMP	better	Lexicon exception
bad + COMP	worse	Lexicon exception

These are design test cases. A final report should record actual pass/fail values from the implementation and explain any mismatch. Irregulars are handled by explicit lexical entries, not by the productive rewrite rules.

TASK 9 – THE PORTER STEMMER (5 MARKS)

9.1 Definitions

The Porter stemmer is a suffix-stripping algorithm for English. A consonant is a letter that is not a, e, i, o or u; y is treated contextually (a consonant after a vowel and a vowel after a consonant). The measure m counts vowel-consonant sequences in the stem pattern [C](VC)^m[V].

Condition	Meaning	Example
v	Stem contains a vowel.	relate has a vowel.
*d	Stem ends with a double consonant.	hopp ends in pp.
*o	Stem ends CVC where final C is not w, x or y.	hop satisfies the condition.

9.2 Worked measure examples

Word	Approximate VC pattern	m (illustrative)
fracture	C C V C C V	2
patients	C V V C V C	2
treated	C C V V C V C	2
infection	V C C V C V C	3
kidneys	C V C C V C	2
swelling	C C V C C V C	2

The exact m calculation is made after the relevant suffix is removed and the remaining stem is segmented into vowel/consonant groups. These examples show the method; a code trace should be used to verify each word.

9.3 Five Porter steps

Step	Main operation	Example
1a	Remove plural endings: sses → ss; ies → i; s removed in defined contexts.	fractures → fracture
1b	Remove -ed / -ing when the stem contains a vowel; apply post-processing such as e insertion or consonant handling.	treated → treat; swelling → swell
1c	Change terminal y to i when the stem contains a vowel.	possibly → possibli (classic algorithm example)
2	Replace longer derivational suffixes with shorter forms when m > 0.	relational → relate
3	Remove/replace selected suffixes when m > 0.	effective → effect
4	Remove a larger suffix set when m > 1.	activation → activ
5	Remove terminal e or one l from ll under the specified m conditions.	rate → rate / controll → control

The classic Porter algorithm is conventionally described as Steps 1a, 1b, 1c, 2, 3, 4 and 5a/5b. They are grouped into five broad stages in some teaching summaries. Implementation should use the canonical NLTK PorterStemmer rather than an informal hand-written approximation.

9.4 Six illustrative word traces

Word	Stemmer result (typical)	Lemmatised form	Comment
fractures	fractur	fracture	Stemmer removes e; lemmatiser returns dictionary lemma.
fractured	fractur	fracture	Both relate to same lemma.
fracturing	fractur	fracture	Progressive ending removed.
patients	patient	patient	Plural suffix removed.
treated	treat	treat	Past form reduced.
nephrectomies	nephrectomi	nephrectomy	Stemmer may not restore correct y.

9.5 Error types and retrieval

Over-stemming occurs when unrelated terms are reduced to the same stem, producing clinically misleading matches. Under-stemming occurs when related forms remain different. A stemmer does not know whether two terms are medically equivalent; a morphological analyser uses a lexicon and features and can preserve distinctions.

Retrieval condition	Precision	Recall	F1	Interpretation
No stemming	Measure on 40 queries	Measure	Calculate	Exact forms only; likely misses inflections.
Porter stemming	Measure	Measure	Calculate	May improve recall but can over-stem.
Full FST analysis	Measure	Measure	Calculate	Can improve controlled conflation; depends on lexicon coverage.

For every query, calculate precision = relevant retrieved / all retrieved; recall = relevant retrieved / all relevant in corpus; F1 = harmonic mean. The 40 query set and relevance judgements are not attached, so real scores must be generated from the completed evaluation set.

TASK 10 – IMPLEMENT AND TEST THE PIPELINE (3 MARKS)

10.1 Python implementation

The following compact implementation demonstrates the main reproducible components using the Python standard library. NLTK's PorterStemmer is used when available. The code should be saved as nlp_hospital.py and run on the documented synthetic/de-identified corpus.

```
import re, time
from collections import Counter
try:
    from nltk.stem import PorterStemmer
    porter = PorterStemmer()
except ImportError:
    porter = None

PATTERNS = {
 'patient_id': re.compile(r'\\bH[0-9]{7}\\b'),
 'date': re.compile(r'\\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\\b'),
 'drug_strength': re.compile(r'\\b[A-Za-z][A-Za-z-]*\\s+[0-9]+(?:\\.[0-9]+)?\\s?(?:mg|mcg|g|ml|IU)\\b', re.I),
 'frequency': re.compile(r'\\b(?:OD|BD|TDS|QID|BID|once daily|twice daily)\\b', re.I),
 'lab_value': re.compile(r'\\b[A-Za-z]{1,8}\\s*[:=]?\\s*[0-9]+(?:\\.[0-
9]+)?\\s?(?:mg/dL|mmol/L|g/dL|U/L|%)\\b', re.I),
 'bp': re.compile(r'\\b(?:[6-9][0-9]|1[0-9]{2}|2[0-4][0-9])/(?:[3-9][0-9]|1[0-4][0-9])\\s?mmHg\\b', re.I),
 'temperature': re.compile(r'\\b(?:3[4-9]|4[0-2])(?:\\.[0-9])?\\s?°?C\\b', re.I),
 'icd': re.compile(r'\\b[A-TV-Z][0-9]{2}(?:\\.[0-9A-TV-Z]{1,4})?\\b'),
 'ward_bed': re.compile(r'\\bWard\\s?[A-Z0-9-]+,?\\s*Bed\\s?[0-9]{1,3}\\b', re.I),
 'contact': re.compile(r'\\b(?:\\+91[- ]?)?[6-9][0-9]{9}\\b'),
 'age': re.compile(r'\\b(?:[1-9][0-9]?|1[01][0-9]|120)\\s?(?:years?|yrs?)\\b', re.I),
 'duration': re.compile(r'\\b[0-9]+\\s?(?:day|week|month|year)s?\\b', re.I)
}
TOKEN_RE = re.compile(r"\\d{1,2}[/-]\\d{1,2}[/-
]\\d{2,4}|\\d+(?:\\.\\d+)?\\s?(?:mg/dL|mmol/L|g/dL|mg|mcg|ml|°C)|(?:[A-Za-z]\\.){2,}|[A-Za-z]+(?:-[A-Za-
z]+)+|[A-Za-z]+|\\d+(?:\\.\\d+)?|[^\\w\\s]", re.I)
ABBR = {'htn':'hypertension','dm':'diabetes mellitus','bp':'blood pressure',
        'hr':'heart rate','sob':'shortness of breath','npo':'nil per os',
        'cbc':'complete blood count','iv':'intravenous','im':'intramuscular',
        'po':'by mouth','prn':'as needed','od':'once daily','bd':'twice daily',
        'tds':'three times daily','qid':'four times daily'}
IRREGULAR = {'children':('child','N','PL'),'men':('man','N','PL'),
             'feet':('foot','N','PL'),'was':('be','V','PAST'),
             'were':('be','V','PAST'),'better':('good','ADJ','COMP')}

LEXICON = {'fracture':'N','patient':'N','kidney':'N','treat':'V',
           'admit':'V','severe':'ADJ','renal':'ADJ','infection':'N'}
def tokenize(text):
    return TOKEN_RE.findall(text)
def normalise(text):
    text = text.lower()
    for short, full in ABBR.items():
        text = re.sub(r'\\b'+re.escape(short)+r'\\b', full, text)
    return text
def analyse(word):
    if word in IRREGULAR:
        lemma, pos, feat = IRREGULAR[word]
        return [(lemma, pos, feat)]
    if word in LEXICON: return [(word, LEXICON[word], 'SG/BASE')]
    for suffix, feat in [('es','PL'),('s','PL'),('ing','PROG'),('ed','PAST')]:
        if word.endswith(suffix) and len(word)>len(suffix)+2:
            stem=word[:-len(suffix)]
            if stem.endswith('e') and suffix in ('ing','ed'): pass
            if stem in LEXICON: return [(stem, LEXICON[stem], feat)]
    return []
def stem(word):
```

```
    return porter.stem(word) if porter else word
def run_record(record):
    t0=time.perf_counter()
    toks=tokenize(record)
    entities={k:v.findall(record) for k,v in PATTERNS.items()}
    norm=normalise(record)
    analyses=[(w,analyse(w)) for w in re.findall(r'[a-z]+',norm)]
    stems=[stem(w) for w in re.findall(r'[a-z]+',norm)]
    return {'tokens':toks,'entities':entities,'normalised':norm,
            'analyses':analyses,'stems':stems,
            'seconds':time.perf_counter()-t0}

if __name__ == '__main__':
    sample=("Patient H1234567 admitted 04/11/2026. BP 120/80 mmHg. "
            "Paracetamol 500mg BD for 3 days.")
    print(run_record(sample))
```

10.2 Test plan

Test group	Positive input	Negative input	Expected result
Patient ID	H1234567	H12345A7	First accepted, second rejected.
Date	04/11/2026	44/19/2026	First shape accepted; calendar check still required.
Drug strength	Paracetamol 500mg	500mg Paracetamol	First matched by stated pattern.
BP	120/80 mmHg	20/300 mmHg	First accepted, second rejected.
FST	fractures	fracturezzz	First analysed as fracture +N +PL; second OOV.
Irregular	children	childs	children uses lexicon exception; childs should not be accepted as standard plural.

10.3 Corpus and measurement plan

Evidence item	Required target	How to produce
Corpus	≥500 de-identified/synthetic records; ≥25,000 tokens; ≥4 record types	Generate or obtain approved records; record provenance and token count.
Pattern inventory	12 entity types	Run each regex; manually adjudicate TP/FP/FN.
Morphology lexicon	≥300 stems; ≥20 inflectional; ≥15 derivational affixes	Build versioned lexicon and affix table.
Query evaluation	≥40 queries with relevance labels	Two reviewers label relevant record IDs and resolve disagreements.
Annotated words	≥200 word forms	Annotate surface form, lemma, POS and features.
Runtime	Per-stage time	Use time.perf_counter; report environment and repeated-run median.

Illustrative output shape (not a claimed execution result): tokens=18; entity matches are listed by pattern; unknown words are counted as OOV; runtime is printed in seconds. Record actual console output and attach screenshots or logs to the final submission.

10.4 Results table and graph

Stage	Items processed	Matched / analysed	Rejected / OOV	Runtime
Tokeniser	Actual token count	Actual token count	0 or discarded punctuation	Measure
Entity patterns	Record characters	Matches per entity type	Unmatched candidates	Measure
FST analyser	Word tokens	Known analyses	OOV tokens	Measure
Porter stemmer	Word tokens	Stemmed tokens	Usually none	Measure

A graph should plot runtime against record length or FST coverage against lexicon size. Generate it from actual run logs; do not use invented measurements as experimental evidence.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE NORMALISATION DESIGN (2 MARKS)

Criterion	No normalisation	Porter stemming	Full FST analysis
Recall	Low for inflected variants	Often higher than exact matching	High for forms covered by lexicon/rules
Precision	High for exact strings, but misses variants	May fall due to over-stemming	Can preserve lexical identity and features
Coverage	No morphology required	Broad English suffix coverage	Limited by lexicon and rule coverage
Runtime	Lowest	Low; linear suffix processing	Linear in input length with finite-state lookup; larger resources
Maintenance	Low	Low	Higher: lexicon, affixes and rules need clinical review
Explainability	Exact string match	Suffix rule can be shown	Analysis and features can be inspected

Recommendation: deploy a hybrid design. Use regular expressions and DFAs for structured entities, an FST for high-confidence clinical morphology, and Porter stemming only as a fallback for unrecognised English terms. Keep the original text and raw token alongside every normalised representation. Do not conflate clinically opposite or merely related terms.

The expected failure modes are unknown abbreviations, out-of-lexicon terms, OCR errors, ambiguous abbreviations and semantic relations that are not morphological. A clinician may notice missing search results or irrelevant matches. The system should expose confidence and source text so that the user can verify the match.

Manual finite-state resources are economical when the domain vocabulary is stable, the rules are explainable and the same forms recur frequently. A learned approach becomes more economical when vocabulary changes rapidly, OCR noise is substantial, code-mixing is frequent and sufficient labelled examples exist to train and maintain a model.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

12A. Impact analysis (2 marks)

The new input introduces OCR character errors, line-break hyphenation, missing diacritics, out-of-vocabulary words, non-English morphology and token-level code-mixing. The original design assumed clean typed English and exact matching against a mostly closed vocabulary. Those assumptions no longer hold.

New input issue	Original component affected	Expected impact
0/O, 1/l, rn/m confusion	Regex and DFA validators	Valid identifiers and values may be rejected; false matches may rise.
Line-end hyphenation	Tokenizer	One word may be split into two tokens.
Lost diacritics	Lexicon/FST	Surface form may not match stored lexical entry.
OOV / misspelling ≥20%	Morphological analyser	Coverage decreases; unknown tokens increase.
Romanised Tamil-English mix	English lexicon and rules	English-only affixes cannot analyse regional morphology.
Code-mixed tokens	Language identification and context	Single-language assumptions fail.

12B. Evaluation of original design (2 marks)

Record A – OCR example: “Pati ent H1234O67 adm1tted 04/11/2026; BP 12O/80 mmHg.” The patient-ID regex rejects the O character in the numeric field; the admission word may be split; the BP regex rejects letter O in the systolic number. The first failure occurs at character-level validation.

Record B – code-mixed example: “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. The English entity regex still recognises paracetamol 500mg, but the English morphological analyser returns no analysis for ku, irundhuchu and kuduthen. The first failure occurs at language/lexicon coverage.

Record C – combined example: “patient-ku fev er irundhuchu; paracetamol 5O0mg kuduthen”. The tokenizer may split fever and the drug-strength regex rejects 5O0mg. Code-mixed tokens are also OOV. This record fails at both OCR correction and language identification.

Measure	Clean baseline	Noisy sample (expected direction)	How to measure
Regex precision/recall	Baseline from Task 2	Usually decreases due to substitutions and malformed spans	Annotate entity spans and calculate TP/FP/FN.
FST coverage	Baseline from Task 7	Decreases as OOV and code-mixed forms rise	Analysed tokens / eligible tokens.
Retrieval F1	Baseline from Task 9	Decreases when query and record forms differ	Evaluate same 40 queries on noisy records.
Runtime	Baseline timing	May increase if correction candidates are explored	Measure with perf_counter.

A minimum 20% OOV/noisy token rate implies that an English-only analyser cannot achieve full coverage even if every in-vocabulary token is handled correctly. Exact numerical degradation must be measured on the three annotated noisy samples and the complete test set.

12C. Adaptation or redesign (2 marks)

Decision: MODIFY the original design. The finite-state layer remains valuable for deterministic clinical formats and known morphology, but it must be preceded by OCR-aware correction and supplemented with language identification and regional lexicon resources.

Component	Change	Reason
Pre-processing	Add OCR confusion map and line-break dehyphenation; retain original text.	Repairs common character-level errors without losing evidence.
Regex patterns	Add tolerant candidate patterns, then validate	Avoids accepting every noisy string as valid.

	corrected candidates with strict patterns.	
Automata	Use weighted/edit-distance candidate matching before strict DFA validation.	Ranks near-matches instead of binary reject-only behaviour.
Lexicon/FST	Add validated Romanised regional forms and continuation classes; mark language per token.	Improves code-mixed coverage.
Rule cascade	Add diacritic-folding and regional spelling variants as optional mappings.	Handles common orthographic variation.
Fallback	Use character n-gram language identification and Porter stemming only for likely English OOV words.	Avoids forcing all tokens through English morphology.

12D. Evidence-based defence (2 marks)

Modification is preferable to retaining the original system because exact matching will reject common OCR errors and English-only analysis will ignore code-mixed tokens. A complete redesign is unnecessary because the existing DFA/FST layer remains accurate and efficient for clean structured values and known clinical terms. The revised system keeps a strict validation stage after fuzzy candidate generation, preventing approximate matching from silently changing patient identifiers, drug strengths or dates.

The main trade-off is higher runtime and maintenance effort. OCR correction may produce several candidates; language identification and regional lexicon expansion require labelled examples and clinician review. Weighted edit-distance should be restricted to low-risk candidate generation. For patient identifiers, dates, drug strengths and doses, the system must require confirmation or strict validation before storing a corrected value.

Remaining limitations: finite-state methods do not understand full clinical intent, resolve all semantic ambiguity, infer an undocumented abbreviation reliably, or model arbitrary nested structures. A clinician-facing system must preserve source text, display uncertainty and support human verification.

Final recommendation

Deploy a hybrid, auditable normalisation pipeline: (1) preserve raw text; (2) perform OCR-aware candidate correction and token-level language identification; (3) validate structured entities with strict regex/DFA rules; (4) analyse known morphology with a clinical FST; (5) use Porter stemming only as a controlled fallback; and (6) rank search results using context and clinical synonym resources. This balances precision, recall, explainability and maintainability.

APPENDIX A – IMPLEMENTATION AND SUBMISSION CHECKLIST

Item	Before submission
Corpus provenance	☐ Confirm 500+ records, 25,000+ tokens and four record types; document synthetic/de-identified source.
Lexicon	☐ Confirm at least 300 stems, 20 inflectional affixes, 15 derivational affixes and separate irregular list.
Annotations	☐ Complete 200 word-form annotations and 40 query relevance judgements.
Automata	☐ Include full diagrams, transition tables, NFA trace, complete subset construction and minimisation counts.
Regex evaluation	☐ Replace all “To measure” cells with TP, FP, FN and metrics from actual corpus.
FST	☐ Provide executable transducer or clearly documented finite-state implementation, all five traces and generation tests.
Porter	☐ Verify six hand traces against the canonical implementation and report agreement/divergence on 200 forms.
Retrieval	☐ Run 40 queries under all three conditions and calculate precision, recall and F1.
Plots	☐ Insert graph generated from actual timing/coverage logs.
Day-4 challenge	☐ Annotate at least three noisy records and measure degradation against clean baseline.
Presentation	☐ Check page numbers, figure/table numbers, spelling, student details and remove blank placeholders.

APPENDIX B – SAMPLE SYNTHETIC RECORD TYPES

Record type	Example content
Discharge summary	Patient ID, admission/discharge date, diagnosis, treatment and follow-up.
Prescription	Drug name, strength, dosage frequency, route and duration.
Operation note	Procedure name, anatomy terms, pre/post-operative diagnosis.
Nursing observation	Temperature, blood pressure, pulse, oxygen saturation and ward/bed.

End of Assignment 1 – CO1.